MARJON SEALZA LIFE & STORE

COMPLETE PROJECT DOSSIER

Consolidated project decisions, architecture, implementation inventory, deployment facts, security requirements, and remaining work

Prepared 20 October 2026 • Edition v1 • Consolidated, non-repetitive reference

This document separates confirmed observations from planned work and unverified assumptions. It is a project reference, not proof that unfinished features have been deployed.

Internal project reference • No payment secrets or credentials included

Contents

1. Executive summary and current status
2. Product vision and user experience
3. Architecture, domains, and deployment
4. Storefront and order journey
5. PayMongo and webhook design
6. Worker API and D1 data model
7. Security and privacy requirements
8. Repository and source-package inventory
9. Verification record and limitations
10. Deployment and release procedure
11. Work plan and acceptance checklist
12. Google AI Studio handoff prompt
13. Reference identifiers and URLs

1. Executive summary and current status

Marjon Sealza LIFE & STORE is one official website with two distinct experiences: LIFE is the first landing page and personal/portfolio experience; STORE appears only after the visitor switches to it. Keep LIFE and STORE content and systems separate while sharing the official domain and consistent branding.

The official website is marjonsealza.life, with www.marjonsealza.life configured. Preserve the existing portfolio and extend it; do not replace it with a duplicate prototype or misrepresent a separate app as the official website.

CURRENT TRUTH: the latest confirmed production Pages deployment is the portfolio site. PayMongo source code is prepared in a separate module, but end-to-end integration with the live storefront has not been verified. Live payments are not approved.

Area	Last known status	Meaning
Official site	Portfolio shell observed at marjonsealza.life; store paths returned same shell	No confirmed public storefront/cart/checkout.
Cloudflare Pages	Project eanuxx; npm run build; output dist	Pages deployment is separate from Worker deployment.
PayMongo Worker	/health previously returned HTTP 200, ready=false, mode=test	Endpoint exists, but readiness was false at that check.
Source tests	60/60 tests passed in source-package node --test run	Not proof of production build or checkout E2E.
D1	Migrations 0001–0003 recorded; no orders/events at last check; placeholder products unavailable	Database groundwork only.
Live payments	Not approved	Stay in test mode until verified and explicitly authorized.

Status terminology

- Prepared means files exist; it does not mean integrated or deployed.
- Tests passed refers only to the observed 60/60 source-package tests.
- A successful Pages deploy does not deploy the separate Worker.
- A browser redirect is not proof of payment; verified backend/webhook state is authoritative.
- Never include secret values in chat, source control, frontend code, or this dossier.

2. Product vision and user experience

Experience rules

- LIFE is the default first landing experience.
- STORE appears only after the visitor switches from LIFE.
- Keep LIFE and STORE content, routes, and business logic distinct; use the same official site and brand.
- Preserve existing portfolio content and navigation unless a deliberate tested change is approved.
- Do not deploy a duplicate site/prototype as a replacement for the official domain.

Store journey

1. Visitor switches from LIFE to STORE and browses available products/services.
2. Visitor views details/prices, adds items to cart, and reviews quantities and totals.
3. Backend validates product IDs, availability, quantity, price, and total; never trust client-supplied prices.
4. Backend creates a pending order and PayMongo Hosted Checkout session in test mode.
5. Customer completes, cancels, or abandons checkout.
6. PayMongo webhook is signature-verified and processed idempotently; order/payment status is updated server-side.

7 Customer status page queries backend order state and displays an accurate result.

Placeholder catalogue

Previously inserted D1 placeholder IDs: digital-consultation, website-planning, and custom-project-inquiry. They were marked unavailable at last recorded check. Do not sell them until copy, pricing, availability, and fulfilment behavior are confirmed.

3. Architecture, domains, and deployment

Component	Identifier / configuration	Notes
Official domain	https://marjonsealza.life	Primary domain.
WWW domain	https://www.marjonsealza.life	Check canonical redirects at release.
Pages project	eanuxx	Project ID 17fe1494-1af2-4107-86b2-cb81abe627ca.
Pages preview	https://eanuxx.pages.dev	Useful for smoke tests.
GitHub repository	marjoncompany-lgtm/marjonsealza	Repository ID previously observed: 1420571021; branch main.
Pages build	npm run build; output dist	Use npm only; no Bun.
Worker	marjonsealza-paymongo	Separate service from Pages.
Worker base	https://marjonsealza-paymongo.emeritabuilder.workers.dev	Worker URL previously used.
Health route	/health	Previously HTTP 200: {service: paymongo-webhook, ready:false, mode:test}.
Webhook route	/webhooks/paymongo	Intended route; verify deployed route before registration.

Latest observed Pages deployment: commit 3a0c47ba13bf327ad7f47530e1d093af6beb4e3d; deployment ID 3b3e9938-b126-4d04-a8fa-918f343f2ec4; dated 15 October 2026; reported uses_functions:false. This indicates Pages Functions were not part of that deployment; the Worker is separate.

Earlier route inspection found /store, /shop, /products, /cart, /checkout, /payment, and /storefront returned the portfolio shell. The separate AppDeploy app eanuxx-emporium-mpouu6 was not the official domain and its Orders view said the payment gateway was not connected. Do not treat it as production storefront.

Current architecture uses Cloudflare D1 for order/payment records. Do not introduce Google Drive or another storage service as the LIFE & STORE database unless separately approved.

4. Storefront and order journey

Catalogue and cart

- Only show products/services marked available and approved for sale.
- Use authoritative server-side product and price data.
- Revalidate product IDs, availability, quantities, and totals at checkout.
- Reject negative, zero, malformed, excessive, or unavailable quantities/items.
- Store line-item snapshots on orders so historical purchases remain readable after catalogue changes.

Checkout pages and order states

- Create checkout sessions server-side only after validation.
- Redirect to the hosted checkout URL returned by the backend.
- Provide success, cancel, failed, or pending messaging as appropriate; success-page visits alone do not confirm payment.
- Use non-guessable order references that do not expose unnecessary personal information.

- Handle refreshes, duplicate submissions, abandoned sessions, delayed webhooks, and duplicate events without duplicate orders or fulfilment.
- Keep payment status separate from fulfilment status.

Admin/store operations

The source package lists protected /api/admin/* routes. Every sensitive operation needs server-side authorization and audit logging. Confirm exact admin operations against the current code before promising particular screens or workflows.

5. PayMongo and webhook design

Chosen direction

Use PayMongo Hosted Checkout rather than a custom card-entry form. Keep the integration in test mode until checkout, webhook delivery, order updates, security, and recovery tests are demonstrated. Live payments require explicit authorization.

Expected processing sequence

1. Customer sends checkout request.
2. Worker validates items and computes total from trusted product records.
3. Worker creates pending order and requests Hosted Checkout session using server-side credentials.
4. Browser receives only safe checkout URL and public order reference.
5. PayMongo sends webhook to Worker after the payment event.
6. Worker verifies signature against the raw request body per current PayMongo docs.
7. Worker deduplicates by event identity and applies a safe state transition.
8. Worker records event processing and updates order/payment state only from verified evidence.
9. Customer-facing page queries backend status.

Webhook configuration

Intended endpoint: https://marjonsealza-paymongo.emeritabuilder.workers.dev/webhooks/paymongo

Confirm the deployed Worker handles this exact route and signature verification is ready before registering it. This URL is the intended target, not proof of readiness.

- Primary event to investigate/enable for Hosted Checkout: checkout_session.payment.paid.
- If available for the account and supported by the code, consider checkout_session.payment.failed.
- Verify exact event names and availability in the current PayMongo dashboard and official documentation; do not assume all accounts expose the same events.
- A browser redirect is not a substitute for a verified payment event.

Worker secrets — names only

Secret name	Purpose	Rule
PAYMONGO_SECRET_KEY	Server-side authentication to PayMongo	Worker secret; never frontend or Git.
PAYMONGO_WEBHOOK_SECRET	Verify webhook signatures	Worker secret for the exact endpoint/mode.
ADMIN_API_TOKEN	Protect admin API where token auth is used	Strong unique random value; never paste into chat.

Set these in Cloudflare Worker Settings → Variables and Secrets. An earlier secret-list inspection returned an empty list, but a later check was blocked, so current configuration is unknown. Verify directly in Cloudflare; do not send secret values in chat.

Official docs: https://docs.paymongo.com/docs/payment-channels-hosted-checkout and https://docs.paymongo.com/docs/developer-tools-webhook-setup-management

6. Worker API and D1 data model

Route / schema	Expected purpose / known state
GET /health	Readiness check; must not expose secrets.
GET /api/products	Public product data for storefront.
POST /api/checkout	Server-side validation and Hosted Checkout creation.
GET /api/orders/:reference	Safe order-status response.
POST /webhooks/paymongo	Webhook receiver and signature verification.
/api/admin/*	Protected administrative API; inspect actual routes in source.
d1_migrations	Migrations 0001–0003 recorded.
store_orders	Added fields: fulfillment_status, customer_name, customer_email, customer_phone, line_items_json, updated_at, paymongo_mode.
payment_webhook_events	Added processed_at.
Other schema	store_products, admin_audit_log, and indexes were recorded.

- Use unique order references and provider event IDs.
- Process webhooks idempotently with safe transactional state changes.
- Store order item/price/currency/total snapshots.
- Record necessary provider references and timestamps without unnecessary sensitive data.
- Never log secrets or card data.
- At last DB verification there were zero orders and webhook events; the three placeholder products were unavailable.

7. Security and privacy requirements

- Keep all payment credentials in Cloudflare Worker secrets; never in frontend code, public environment variables, screenshots, prompts, or GitHub.
- Verify signatures using PayMongo's documented process and the raw body; reject invalid signatures.
- Deduplicate webhook deliveries and audit payment/order transitions.
- Authenticate and authorize every admin API operation server-side.
- Validate user-controlled fields, rate-limit where appropriate, and return safe errors.
- Do not expose stack traces, database bindings, private customer data, or secrets in public responses.
- Use HTTPS and least-privilege account access.
- Keep test mode until test evidence is reviewed; never enable live mode without explicit approval.
- Do not delete or overwrite existing production content, data, domains, or Workers during integration.

8. Repository and source-package inventory

Development rules

- Repository: marjoncompany-lgtm/marjonsealza; production branch main.
- Standardize on npm only; no Bun and no Bun lockfiles.
- Keep package.json and package-lock.json synchronized; use npm ci where appropriate.
- Run the project test command and npm run build before declaring ready.
- Cloudflare Pages build output: dist.

- Fix build errors before deploy and report test, build, Pages deployment, and Worker deployment separately.

Latest prepared archive

Marjon_Sealza_LIFE_Store_Complete_Codebase_v20_PayMongo_Code_Prepared_Not_Integrated.zip

The v20 archive was based on v19 and added a separate v19/paymongo-module/ folder with copied Worker implementation files, migration SQL 0001–0003, focused tests, README, .env.example, and PAYMONGO_STATUS.txt. Root package version was bumped to 2.0.0. This separation means code is prepared but not confirmed wired into the storefront.

Observed source test result: node --test reported 60/60 passing after a compatibility test was fixed by adding official documentation links to README. This does not prove the current repository's production build passes, that the deployed Worker is current, or that a PayMongo checkout completes end-to-end.

9. Verification record and limitations

Previously observed

- Cloudflare Pages project and domain mapping were observed.
- Pages deployment succeeded but reported uses_functions:false.
- Worker /health returned HTTP 200 with ready=false and mode=test.
- D1 migrations 0001–0003 and store/payment schema were recorded.
- Last DB check: zero orders and webhook events; placeholder products unavailable.
- v20 ZIP integrity passed; source tests reported 60/60 passing.

Not yet demonstrated

- Production storefront/cart/checkout on marjonsealza.life.
- Successful PayMongo test checkout end-to-end.
- Valid webhook received and accepted by currently deployed Worker.
- Verified webhook updating an order and duplicate delivery being safely ignored.
- Current target repository's production build after integration.
- Fully tested admin/order-status experience.
- Live payment readiness or authorization.

A previous GitHub write integration attempt returned HTTP 403: Resource not accessible by integration. Do not claim ChatGPT pushed a commit or deployed the repository through that path. Google AI Studio may have repo access, but its commit, build, and deployment outcomes need evidence.

10. Deployment and release procedure

1. Confirm exact repository, branch, and current deployed commit; compare the existing repo to v20 rather than blindly replacing it with an old ZIP.
2. Have Google AI Studio finish integration in the existing repository without creating duplicate website, database, or Worker.
3. Review the diff for unrelated rewrites, exposed secrets, duplicate systems, and regressions.
4. Run npm ci, the documented tests, and npm run build; save actual outputs.
5. Confirm Worker environment and D1 binding PAYMENTS_DB are configured in the correct environment.
6. Set test credentials and webhook signing secret in Cloudflare Worker secrets; never commit values.
7. Deploy Worker separately from Pages; check /health and confirm readiness matches configuration.
8. Register test webhook only after route and signature verification are ready.

9 Complete a test checkout; inspect PayMongo delivery logs and verify the matching order state.

10 Test duplicate webhook, invalid signature, cancellation, failed/abandoned checkout, unavailable product, changed price, invalid quantity, and unknown order reference.

11 Deploy Pages after build and regression tests; verify both official domains and LIFE/STORE navigation.

12 Keep live mode disabled until evidence is reviewed and the user explicitly authorizes it.

Required release report

- Commit SHA and branch; files changed.
- Exact test command and pass/fail result.
- Exact npm run build result.
- Pages deployment ID/URL and Worker deployment status separately.
- Webhook route, subscribed events, and delivery result — never the signing secret.
- Database migration status and test-data cleanup plan.
- Remaining risks and unfinished items; no unverified claims.

11. Work plan and acceptance checklist

Phase	Work	Acceptance evidence
A — Audit	Compare current repo with prepared module and inspect routes.	Diff reviewed; no duplicate site/DB/Worker; npm locks consistent.
B — Code	Checkout validation, session creation, order lifecycle, signature verification, idempotency, status pages.	Tests cover success and failure paths.
C — Build	Run tests and npm run build.	Actual outputs recorded; no ignored failures.
D — Config	Set Worker secrets and confirm D1 binding.	Readiness true only with required config; no secret exposure.
E — Webhook	Register endpoint and supported Hosted Checkout events.	Delivery succeeds; invalid signatures rejected.
F — E2E	Test payment, duplicate event, failure/cancel cases.	Correct state transitions; no duplicate fulfilment.
G — Site	Add LIFE-to-STORE switch and store routes on existing site.	Portfolio regression tests pass; no duplicate prototype.
H — Release	Deploy Pages and Worker separately; smoke-test.	Both deployment versions recorded; no critical failures.
I — Live decision	Review test evidence and obtain explicit approval.	Live mode only after authorization.

Definition of done

- LIFE portfolio still works and is the first landing page.
- STORE is accessible by switching to it and remains separate in content/system.
- Server validates product/price/quantity and creates Hosted Checkout in test mode.
- Valid webhooks update orders; invalid and duplicate events are safely handled.
- Customer pages show backend-confirmed order state.
- Admin APIs are protected and audited.
- Tests and npm run build pass; Pages and Worker are separately verified.
- Live payments remain disabled unless explicitly authorized.

12. Google AI Studio handoff prompt

Finish the PayMongo integration code in my existing GitHub repository for marjonsealza.life. Use PayMongo Hosted Checkout and the existing Cloudflare Worker and D1 database architecture. Do not create a duplicate website, database, or Worker. Preserve the existing site structure: LIFE is the first landing page, and STORE appears only after the visitor switches to it. Keep LIFE and STORE content and systems separate, and preserve current production features. Implement secure server-side checkout session creation, server-side order and price validation, webhook signature verification, idempotent payment processing, order/payment status updates, and customer success/cancel/failure or pending pages. Confirm exact PayMongo event names against current official documentation and the account dashboard. Use secret names only. Never request, print, or commit secret values. Keep everything in test mode; do not enable live payments. Use npm only. Keep package.json and package-lock.json synchronized, do not use Bun, and remove Bun lockfiles if any. Run the automated tests and npm run build; fix failures. Cloudflare Pages and the Cloudflare Worker are separate deployments. Report each separately. If Cloudflare access is unavailable, do not claim deployment succeeded; finish the code and list remaining deployment steps. At completion report commit SHA, changed files, test command/result, build result, Pages deployment result, Worker deployment result, webhook route and supported events, and remaining risks. Do not call anything deployed unless verified.

13. Reference identifiers and URLs

Reference	Value
Official website	https://marjonsealza.life
WWW website	https://www.marjonsealza.life
Pages preview	https://eanuxx.pages.dev
GitHub repo	https://github.com/marjoncompany-lgtm/marjonsealza
Worker base	https://marjonsealza-paymongo.emeritabuilder.workers.dev
Health	https://marjonsealza-paymongo.emeritabuilder.workers.dev/health
Webhook target	https://marjonsealza-paymongo.emeritabuilder.workers.dev/webhooks/paymongo
Hosted Checkout docs	https://docs.paymongo.com/docs/payment-channels-hosted-checkout
Webhook docs	https://docs.paymongo.com/docs/developer-tools-webhook-setup-management
Pages project / ID	eanuxx / 17fe1494-1af2-4107-86b2-cb81abe627ca
Pages build / output	npm run build / dist
D1 binding	PAYMENTS_DB
Worker secret names	PAYMONGO_SECRET_KEY; PAYMONGO_WEBHOOK_SECRET; ADMIN_API_TOKEN
Prepared code ZIP	Marjon_Sealza_LIFE_Store_Complete_Codebase_v20_PayMongo_Code_Prepared_Not_Integrated.zip
Earlier plan PDF	Marjon_Sealza_LIFE_Store_Master_Plan_v4_PayMongo_To_Do.pdf

This dossier consolidates available project decisions and status notes into one non-repetitive reference. It does not replace the repository, contain credentials, or constitute proof of deployment.